

How the RREA Model Is Built on WarpX/AMReX: A Beginner-Friendly Architecture Guide

RREA WarpX/AMReX project

Abstract

This guide explains, from first principles, how the RREA (Relativistic Runaway Electron Avalanche) and **relativistic feedback** simulation model is constructed on top of two open-source frameworks: **AMReX** (a block-structured mesh and particle library) and **WarpX** (a particle-in-cell plasma code built on AMReX). The model tracks not only the forward electron avalanches but also the backward-propagating bremsstrahlung photons and pair-produced positrons that can re-seed new avalanches at the source region — Dwyer’s relativistic feedback mechanism, which turns isolated avalanches into a self-sustaining discharge and is essential for Terrestrial Gamma-ray Flash physics. The guide is written for a reader who knows basic plasma/atmospheric physics and some C++, but who has never used WarpX or AMReX. It describes what the frameworks provide out of the box, exactly where and how the RREA code attaches to them, what happens during one simulation time step, and why the project is organized the way it is. It documents the schema-6 transport and Maxwell TM engine.

Contents

1	What the Model Simulates	3
2	The Building Blocks: AMReX and WarpX in Five Minutes	4
2.1	AMReX	4
2.2	WarpX	5
3	The Big Picture: Overlay + Native Library	5
4	How the Code Attaches to WarpX	6
4.1	The source-preparation step	6
4.2	The ordering contract	7
5	Species and the Input Deck	8
6	One Time Step, Start to Finish	10

7	The Schema-6 Interaction Tables	12
7.1	One table set for every altitude: density scaling	13
7.2	Loading and integrity checks	14
7.3	Geant4-derived transport data and validation	14
8	The Low-Energy Fluid and the Field Solve	14
8.1	Fluid state	14
8.2	Why the field solve is different from stock WarpX	15
8.3	The Maxwell TM branch	15
8.4	The imposed background and its lateral extent (radial taper)	16
9	Diagnostics, Video, and Checkpoints	16
10	From Simulation to Radio, Longitudinal-Field and Optical Observables	17
10.1	EM fields from the recorded Huygens surface	17
10.2	Radio scope	18
10.3	Optical emissions at 337.1 nm and 777.4 nm	19
11	Model Scope and Known Model-Form Uncertainties	21
12	Checking the Physics: Avalanche Multiplication vs. Coleman–Dwyer	23
13	Fail-Closed Operation and Focused Validation	24
14	Physics-Module Index	25
15	Repository Map	33
16	Glossary	34

How to read this guide. If you are new to the project, read Sections 1–2 for the physics goal and a five-minute tour of AMReX and WarpX, then Sections 3–5 for how the code attaches to WarpX and what the input deck controls. The heart of the guide is the walkthrough of one simulation time step (Section 6); nearly every later section expands one box of its data-flow figure. The deeper dives — the interaction tables (Section 7), the avalanche-length benchmark (Section 12), and the fluid and field solve (Section 8) — can be read independently, as can the diagnostics and observable-signature sections. Recurring terms are collected in the Glossary at the end.

1 What the Model Simulates

A thundercloud can sustain quasi-static electric fields above the *runaway breakeven* threshold. In such fields, an energetic electron (MeV scale) gains more energy from the field than it loses to air, so it “runs away,” and its hard collisions knock out further energetic electrons: a Relativistic Runaway Electron Avalanche. The avalanche emits bremsstrahlung gamma rays (the mechanism behind Terrestrial Gamma-ray Flashes), creates positrons, and — crucially — also deposits an enormous number of *low-energy* electrons and ions that make the air conductive. That conductivity discharges the very field driving the avalanche, so the system is self-limiting.

On top of the single-avalanche picture sits **relativistic feedback** (Dwyer): some of the avalanche’s bremsstrahlung photons Compton-backscatter or propagate upstream, and some of its pair-produced *positrons* — whose charge is positive — are accelerated *backward* along the field. When such a backward-moving photon or positron reaches the region where the avalanches start, it can create a fresh energetic electron there (by Compton scattering / photoelectric absorption, or by the positron’s hard Bhabha collisions), seeding a brand-new avalanche. If, on average, each avalanche seeds at least one successor, the discharge becomes self-sustaining and grows explosively until the field collapses. Whether a thundercloud crosses that feedback threshold is one of the central physics questions this model exists to answer, which is why photons and positrons are tracked as full kinetic species rather than treated as passive losses (Figure 1).

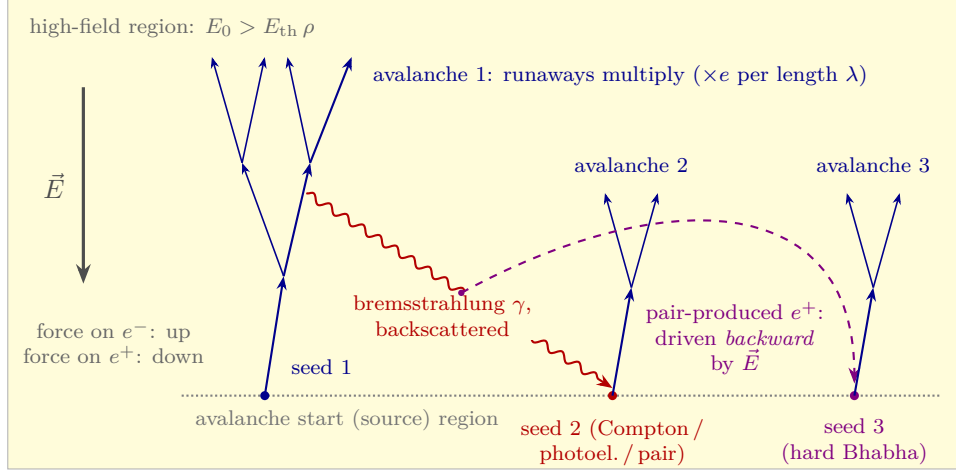


Figure 1: The relativistic feedback loop (Dwyer) the model is built to capture. A seed electron runs away and multiplies into avalanche 1. Some of its bremsstrahlung photons propagate or Compton-backscatter *against* the avalanche direction and liberate fresh energetic electrons near the start region; positrons pair-produced by those photons carry positive charge and are therefore accelerated *backward* by the same field, seeding further electrons through hard Bhabha collisions. If each avalanche seeds on average at least one successor, the discharge is self-sustaining.

Modeling this therefore needs three coupled ingredients:

1. **High-energy particle transport** (electrons, photons, positrons, $\gtrsim$ keV–MeV–GeV): individual particles pushed through air with Monte Carlo collisions. Photons and positrons get the same first-class treatment as electrons precisely because they carry the relativistic feedback.
2. **A low-energy plasma fluid** (electrons below a cutoff, positive and negative ions): far too numerous to track as particles; represented as densities on the mesh, evolving by attachment, recombination, and drift, and producing a conductivity σ .
3. **Self-consistent field feedback**: the electric field is the imposed thundercloud profile *relaxed* by the conductivity and by the space charge of both populations.

The RREA model implements (1) as a WarpX particle extension, (2) and (3) as a native AMReX library, and couples them inside the WarpX time step. The simulation geometry is cylindrical RZ (radius r , altitude z), which matches the natural symmetry of a vertical avalanche column.

2 The Building Blocks: AMReX and WarpX in Five Minutes

2.1 AMReX

AMReX is a C++ framework for block-structured mesh codes. The concepts used constantly in this project:

- **Box / BoxArray:** the rectangular index space of the mesh is chopped into boxes (here up to 64×64 cells, set by `amr.max_grid_size`). Boxes are the unit of parallel distribution.
- **DistributionMapping:** assigns each box to one MPI rank.
- **MultiFab:** “multiple FArrayBoxes” (Fortran array boxes) — *the* AMReX data structure: one floating-point field (e.g. electron density) stored box-by-box across all ranks, with ghost cells for stencils. The fluid, charge, and WarpX-facing field arrays are MultiFabs; the Maxwell update uses a replicated staggered state assembled from them.
- **ParticleContainer:** distributed particle storage; each particle carries position, momentum, a statistical *weight* (how many real particles one computational “macroparticle” represents), and optional extra components.

2.2 WarpX

[WarpX](#) is an electromagnetic/electrostatic particle-in-cell (PIC) code built on AMReX. A PIC code advances plasma in a loop: deposit particle charge onto the mesh $\rightarrow$ solve for fields $\rightarrow$ interpolate (“gather”) fields to particles $\rightarrow$ push particles. WarpX provides, ready-made:

- the RZ cylindrical geometry and distributed particle storage;
- named particle species defined in a text input deck (`particles.species_names=...`), each backed by a ParticleContainer;
- the main time loop (`WarpXEvolve.cpp`), field containers, I/O, checkpointing scaffolding, and MPI orchestration;
- an electrostatic solver, `ComputeSpaceChargeField`, which the RREA model *replaces* (Section 4).

The project gitlinks pin **WarpX 26.06** and **AMReX 26.06** in protected archival forks. AMReX is built from source inside the WarpX build; the executable is RZ, double precision, MPI+OpenMP.

The key idea to keep in mind: *WarpX supplies particles, mesh, and the time loop; the RREA code supplies the physics of air and the field solver, and inserts itself at a few precisely chosen points of the loop.*

3 The Big Picture: Overlay + Native Library

The protected forks archive pristine release commits; this repository carries no modified copy of either framework. The RREA implementation consists of two parts:

Native library A standalone static library (`rrea_amrex_field_solver`; sources in `native_amrex_rrea/src` and `native_amrex_rrea/include/rrea/`) that depends only on AMReX — not on WarpX. It contains the fluid state, the Maxwell advance, the field initializer and atmosphere profile, the interaction-table loader, the seed schedule, and the event accumulator.

Because it has no WarpX dependency, this physics can be built and tested without constructing WarpX. Focused MPI variants additionally exercise distributed invariants.

WarpX overlay (`native_amrex_rrea/warpX_overlay/Source/Rrea`) The WarpX-aware kinetic-transport and coupling layer. It owns the particle loops and their shared final-state primitives, links the native library, and is copied into a WarpX source tree at build time.

Native library (AMReX only)	Overlay (WarpX-aware transport/coupling)
LowEnergyFluidState	RreaParticleInteraction (MC engine)
RreaAmrexAdvance	TransportStep (the four species phases)
RreaFieldInitializer, RreaAltitudePro file	RreaRng (counter-based RNG)
RreaInteractionTables (schema-6 loader)	checkpoint rank-state helpers
RreaEventAccumulator, RreaSeedSchedule	

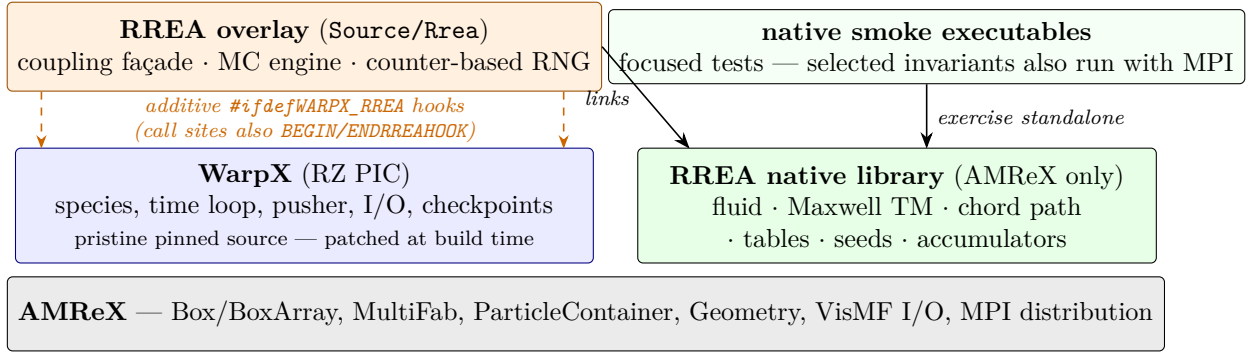


Figure 2: Where each piece of code sits. WarpX and the RREA native library both stand directly on AMReX; the overlay is the only WarpX-aware code and is grafted onto a pristine WarpX tree at build time through guarded, exact-match, idempotent patches (Section 4). The native library is exercised standalone by the native smoke executables before it ever runs inside WarpX; the production executable `warpX.rz` links all of it into one binary.

This split follows dependency ownership (Figure 2): AMReX-only mesh, field and table code lives in the native library; WarpX particle transport and coupling live in the overlay. Header primitives keep much of the transport physics independently testable without running WarpX.

4 How the Code Attaches to WarpX

4.1 The source-preparation step

A pristine WarpX checkout is never edited by hand. The script `scripts/prepare_rrea_warpX_source.py` builds the combined source tree:

1. copies the pristine WarpX tree to a build location (it refuses to run in-place);
2. copies the overlay directory in as `Source/Rrea`;
3. applies a small set of guarded patches to stock WarpX files. Every insertion is `#ifdefWARPX_RREA`-guarded; each call-site hook is additionally wrapped in `//BEGINRREAHOOK...//ENDRREAHOOK` comments. Each patch is applied by an exact-match text search (“needle”). The patch operations are idempotent, so `--refresh` can update an existing prepared tree; if a WarpX update moves a needle, the script *aborts* instead of guessing.

The patches add guarded declarations and calls without deleting stock behavior. The full list of patched files:

WarpX file	What the patch adds
<code>CMakeLists.txt</code>	<code>add_subdirectory(Source/Rrea)</code> ; overlay sources join the WarpX library target with <code>WARPX_RREA=1</code> .
<code>Source/WarpX.H</code>	a <code>SpaceChargeSolvePurpose</code> enum and a <code>purpose</code> parameter on <code>ComputeSpaceChargeField</code> , so the hook can distinguish initialization solves from evolution solves.
<code>Source/FieldSolver/WarpXSolveFieldsES.cpp</code>	the field-solver replacement: at the top of <code>ComputeSpaceChargeField</code> , call <code>RreaWarpXCoupling::BeforeFieldSolve(...)</code> ; if it returns true, WarpX’s own electrostatic solve is skipped entirely.
<code>Source/Evolve/WarpXEvolve.cpp</code>	three calls in the step loop: <code>InjectScheduledSeeds</code> (before the push), <code>CaptureRreaPositionsBeforePush</code> (just before <code>OneStep</code>), and <code>AfterParticlePushBeforeBoundaries</code> (just after).
<code>Source/Initialization/WarpXInitData.cpp</code>	tags the startup field solve with <code>Initialization</code> purpose.
<code>Source/Diagnostics/FlushFormats/FlushFormatCheckpoint.cpp</code> / <code>Source/Diagnostics/WarpXI0.cpp</code>	write/read the extra RREA checkpoint payload alongside WarpX’s own checkpoint.

4.2 The ordering contract

The physics is only correct if the hooks run in a specific order inside one step (Figure 3): *inject seeds* → *snapshot particle state* → *stock WarpX step* → *RREA trajectory reconstruction and air transport* → *WarpX boundary handling* → *RREA field solve*. The preparation script literally asserts this ordering when patching `WarpXEvolve.cpp`: if a future WarpX version reorders its loop, source preparation fails loudly instead of silently producing wrong physics.

The order is a physics contract, not a convenient insertion point. RREA must see the pushed chord *while an escaping parent still exists*: a particle that leaves the domain this step still traveled its chord through air inside it, and WarpX’s boundary handling would delete it first. At the other end, generic resampling must not run before the atomic secondary groups have been committed — which is why the asserted anchors include WarpX’s own `doResampling`, a stage that runs inside the protected window and is not drawn in Figure 3. This “fail-closed” habit — abort rather than improvise — recurs everywhere in the project (Section 13).

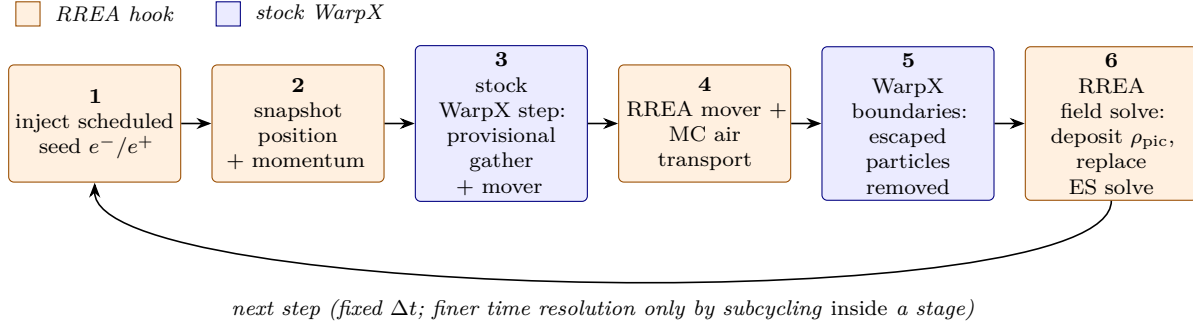


Figure 3: The per-step ordering contract. Orange stages are RREA hooks; blue stages are stock WarpX. The source-preparation script asserts this sequence — plus WarpX’s own `doResampling` between stages 4 and 5, not drawn — when patching `WarpXEvolve.cpp`, and aborts if a WarpX update reorders its loop. The numbers match the walkthrough in Section 6.

5 Species and the Input Deck

The model manages three WarpX species, declared in a completely ordinary WarpX input deck (template: `cases/rrea/production/inputs`):

```

particles.species_names = rrea_electrons rrea_photons rrea_positrons
rrea_electrons.species_type = electron
rrea_photons.species_type = photon
rrea_positrons.species_type = positron

```

All three use `injection_style=none`: WarpX never injects these particles itself. The RREA code owns creation and physical terminal decisions; WarpX’s boundary pass removes escaped slots afterward. At startup the coupling verifies that each named species really has the mass/charge it expects and that the three names are distinct — another fail-closed check.

Everything else the model needs arrives as `rrea.*` input parameters. The important groups:

Group	Representative parameters
Ownership	<code>rrea.enable=1</code> and <code>rrea.field_model=maxwell_rz_yee_material_coupled</code> . External particle fields are forbidden — the field may come only from the RREA solver.
Transport tables	<code>rrea.interaction_table_config</code> (path), <code>rrea.transport_model</code> (exact string match against the transport configuration); photon feedback and positron transport are unconditional and have no deck keys — the table loader aborts unless the transport configuration declares both enabled. Optical-depth guards (<code>rrea.transport_max_optical_depth_per_substep</code> , <code>rrea.transport_max_subcycles</code> , <code>rrea.transport_strict_guards=1</code>).
Population control	<code>rrea.population_ceiling_policy=adaptive_resample_v1</code> (mechanism and the scope of its unbiasedness in §13) with <code>rrea.population_target_electron_macros</code> , <code>rrea.max_electron_macros</code> , <code>rrea.max_photon_macros</code> , <code>rrea.max_positron_macros</code> as fail-closed backstops above the control band; <code>strict_abort_v1</code> is the engine default and the policy of an explicit reference run; <code>rrea.low_energy_cutoff_eV</code> overrides the bundle’s charged floor for a study (§11).
Field / Maxwell	<code>rrea.acceleration_field_v_per_m</code> , <code>rrea.acceleration_z_max_m</code> (imposed profile), mesh spacing and timestep.
Atmosphere	<code>rrea.air_density_model</code> (a constant density ratio at a chosen altitude, or a full altitude profile), the density ratio ρ/ρ_0 itself. The air composition is fixed by the tables’ G4_AIR reference, not by deck fractions. See Section 7.1 for how one table set serves every altitude.
Seeding	<code>rrea.seed_schedule_path</code> (a CSV of when/where/how many seed electrons and positrons appear, at what energy, weight, and direction; the optional <code>species_id</code> column routes each event to its container), <code>rrea.rng_seed</code> . The production default schedule is the PARMA cosmic-ray e^-/e^+ source with its $t = 0$ stationary bath (§14, Deposition and population); the mono-1-MeV pulse and the C&D cohort remain selectable through <code>config/rrea_defaults.json</code> / the driver.
Diagnostics	<code>rrea.diag_interval</code> , <code>rrea.output_dir</code> , <code>rrea.video_diag_*</code> (Section 9).

The *macroparticle weight* deserves a beginner’s note: one computational particle represents w real

particles (w is set by the seed schedule; secondaries inherit the parent scale). Larger w means fewer macros (cheaper, noisier), smaller w the reverse.

One more setting a newcomer should know about: the *time step* is fixed for the whole run (`warpx.const_dt`). There is no adaptive global stepping — wherever finer time resolution is needed, the relevant layer subcycles internally (the Monte Carlo transport under its optical-depth guard; the coupled fluid/Maxwell advance under its conductivity, diffusion and drift limits), keeping the outer step sequence shared by every coupled subsystem and checkpoint.

6 One Time Step, Start to Finish

Figure 4 shows the whole per-step data flow at a glance; the numbered walkthrough below follows the same arrows in execution order.

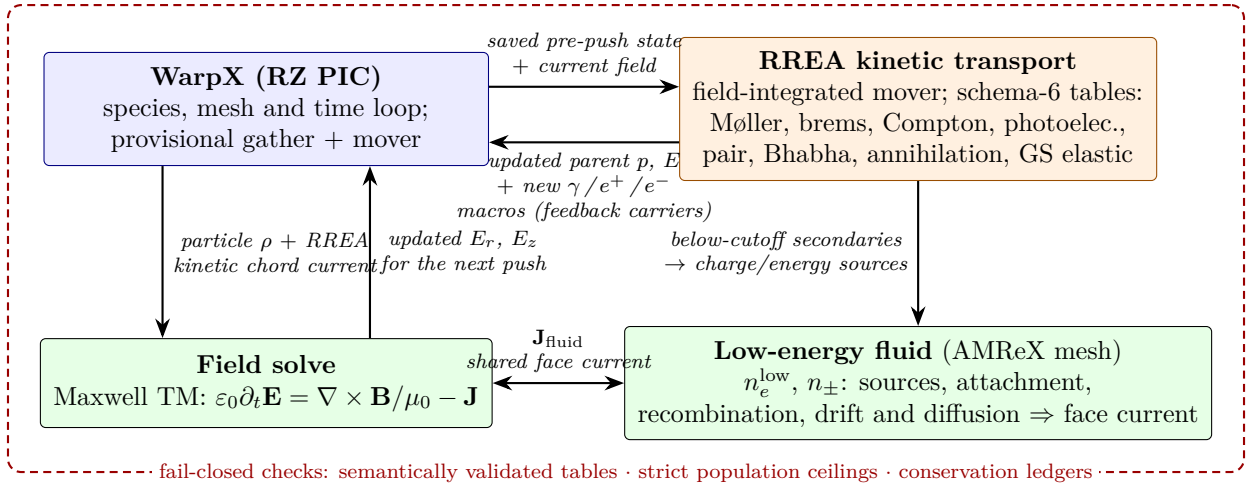


Figure 4: Per-step data flow. WarpX executes its stock step provisionally; the RREA engine reconstructs the physical trajectories from the saved state, applies air physics, and overwrites the endpoints, promoting above-cutoff secondaries to new macroparticles — which closes Dwyer’s relativistic feedback loop — and demoting below-cutoff secondaries into the low-energy fluid; the fluid and kinetic chord currents drive the Maxwell TM update, whose field feeds the next particle push. The dashed frame is the fail-closed checks around the coupled path (Section 13); the execution order within one step is the hook sequence of Figure 3.

All coupling state lives in one singleton, `rrea::warpx::RreaWarpXCoupling`, declared in `native_amrex_rrea/warpx_overlay/Source/Rrea/RreaWarpXCoupling.H`. Its members are defined across sibling translation units, one responsibility each: the field step (`RreaWarpXCoupling.cpp`), startup contracts and table/schedule loading (`RreaCouplingConfig.cpp`), population control (`RreaCouplingPopulation.cpp`), seed and event sources (`RreaCouplingSources.cpp`), the reduced CSV (`RreaCouplingDiagnostics.cpp`), video frames (`RreaCouplingVideoFrame.cpp`), the C&D plane stream (`RreaCouplingPlaneWriter.cpp`), checkpoint/rank-state I/O (`RreaCouplingCheckpoint.cpp`) and the charge-continuity fixture (`RreaActualContinuityFixture.cpp`). Shared helpers sit in `RreaCouplingDetail.H`; the one rank-local sweep over live macroparticles (`RreaForEachValidParticle`, `RreaParticleSweep.H`) gives the soft-deleted-slot guard a

single owner. Its per-step entry points fire in this order:

1. **InjectScheduledSeeds** — reads the seed-schedule CSV and creates any seed macros whose injection time falls in this step, routed by the schedule’s `species_id` into the `rrea_electrons` or `rrea_positrons` container.
2. **CaptureRreaPositionsBeforePush** — copies every managed particle’s position *and momentum* into extra particle components (`rrea_prev_*`): the pre-push state the transport below re-integrates from.
3. **WarpX OneStep** — the stock WarpX gather and relativistic push still execute, but the kick is *discarded*: the engine rebuilds direction, energy and geometry from the captured pre-push state, drives its own running chord with the field impulse re-applied per substep (Strang split, Gauss–Legendre line-integral work), and overwrites WarpX’s endpoint with the point the particle actually reached. `algo.particle_pusher` is therefore inert plumbing; `algo.particle_shape` is *not* (the hook’s own charge deposition uses it).
4. **AfterParticlePushBeforeBoundaries** — the material physics. For each managed particle, the Monte Carlo engine (`RreaParticleInteraction::Apply`) drives the running chord through air from the pre-push state. `Apply` only builds the per-step context (`TransportStep`, `RreaTransportStep.H`) and runs four phases, one translation unit each: the electron, photon and positron loops (`RreaElectronTransport.cpp`, `RreaPhotonTransport.cpp` and `RreaPositronTransport.cpp`), then the commit tail (`RreaSecondaryCommit.cpp` and `RreaNewbornAdvance.cpp`). The charged loops are separate by *policy* only; the drag, elastic and final-state physics is shared (`RreaChargedFinalStates.H`, `RreaPhotonFinalStates.H`, `RreaMaterialPath.H`, `RreaPathFieldWork.H`). Along the chord:
 - continuous energy loss (collisional + radiative drag) and field work are integrated along the chord;
 - discrete interactions compete as hazards: the traversed *optical depth* in each hard channel (Møller, bremsstrahlung, photon Compton / photoelectric / pair production, positron Bhabha / annihilation) is accumulated, and the engine samples which event, if any, occurs and where on the chord. Goudsmit–Saunderson elastic deflection is accumulated separately over each sub-interval;
 - sampling uses the **schema-6 tables** (Section 7) with log-log interpolation, local air-density scaling (Section 7.1), and *forbidden* extrapolation;
 - secondaries above the cutoffs become new macroparticles of the appropriate species; charged secondaries *below* the cutoff are “demoted”: their charge and energy are deposited into mesh accumulators as low-energy-electron and ion sources (cutoffs: §11);
 - every state-changing discrete interaction closes a charge and energy ledger; a violation aborts;
 - randomness comes from a counter-based RNG keyed on the physical identity of the particle, so each particle’s random stream does not depend on MPI layout;
 - before any group of new particles is committed, all ranks plan exact-CIC control over survivors plus staged charged candidates, then collectively preflight both charged plans and all population ceilings (Section 13).
5. **WarpX boundary handling** — stock WarpX removes particles that left the domain.

6. **BeforeFieldSolve** — called where WarpX would solve its electrostatic field; returns **true**, so WarpX’s solver never runs. Instead:

- the hook deposits the energetic particles’ charge itself — in the electrostatic branch stock WarpX pushes with deposition skipped, because the stock design deposits inside the very solver this hook replaces;
- the accumulated event sources (low-energy electrons, positive and direct negative ions, ionization energy) are normalized and handed to the fluid;
- **RreaAmrexAdvance::AdvanceOneStep** evolves the fluid and the field (Section 8);
- the resulting E_r, E_z are copied back into WarpX’s field MultiFabs (so the next step’s particle push feels the updated field);
- video frames and the reduced diagnostics row are written if due.

7 The Schema-6 Interaction Tables

Process rates and several energy/final-state distributions come from pre-generated data compared with Geant4 10.7.4 and organized under **rrea_transport_tables/**. Runtime analytic models remain explicit where used (notably free Klein–Nishina Compton and bounded Modified–Tsai bremsstrahlung angles). The table directory is selected through **transport_physics.json** and the loader checks units, ranges and cutoffs; focused tests compare the physical holdouts.

The shared electron/positron brems angle uses the Modified–Tsai distribution of **G4ModifiedTsai** in the pinned Geant4 source: its Gamma-mixture variable u is conditioned on $0 \leq u \leq 2\gamma$, then mapped by $\cos \theta = 1 - u^2/(2\gamma^2)$, where $\gamma = 1 + K/(m_e c^2)$. This analytic angle is independent of the tabulated photon energy; it does not claim option-4’s energy–angle correlation. It requires no angular table.

- **Electrons:** restricted collisional drag, radiative and soft-radiative drag, hard Møller mean free path + secondary spectrum, bremsstrahlung MFP + photon energy CDF + analytic emission angles, Goudsmit–Saunderson elastic multiple-scattering angle CDFs;
- **Photons:** modeled-channel interaction MFP and probabilities, photoelectric subshell CDFs (with binding energies), nuclear and triplet pair production with Geant4 energy-sharing CDFs. Compton is the one channel whose final state is *not* tabulated: its rate carries Geant4’s incoherent scattering function (measured -4.4% against free Klein–Nishina at 50 keV), but the scattering angle is sampled at runtime from analytic free-electron Klein–Nishina;
- **Positrons:** *restricted* collision stopping (numerically equal to the electron restricted table to 0.3%), soft-radiative drag, hard Bhabha, positron bremsstrahlung, in-flight and at-rest annihilation.

The photon and positron channels are what close the relativistic feedback loop, and the engine makes that unconditional: every *explicitly produced* bremsstrahlung photon is a tracked particle able to Compton-scatter or pair-produce anywhere in the domain (including *behind* the avalanche front), and pair-produced positrons — which the ambient field accelerates backward — are transported with their own drag, hard-Bhabha, bremsstrahlung and annihilation physics. There are no deck

keys for either (§5), so feedback is not a bolted-on approximation (no “feedback factor”): it *emerges* from the same per-particle Monte Carlo transport as the primary avalanche.

Coherent Rayleigh scattering is deliberately excluded from the schema-6 kinetic gamma-ray transport: it has no runtime channel and no rate or angular table. Near-cut photon mean free paths, angular transport, and feedback predictions are conditional on that scope. This does not describe the separate optical CloudScat calculation, whose atmospheric Rayleigh channel is enabled unless explicitly disabled.

“Every explicitly produced” is meant literally, and the qualifier matters. Bremsstrahlung photons below the tracked cut are not created as particles at all; they are carried as continuous radiative drag on the parent. That cut is $k \geq \text{brems_absolute_cutoff_eV}$, a flat energy declared by the loaded transport configuration — the engine binds to it and pins no value, so quote the number from the bundle a result came from. Energy is conserved — the sub-cutoff loss is booked as **electron_soft_radiative_stopping** and is correctly *not* converted into ion pairs — but the photon *count and spectrum* are incomplete below the cut, where the tracked population is only Compton-degraded photons. Any statement about photometer counts, low-energy irradiance, or the backscattered feedback carriers (which pile up below $m_e c^2/2 = 255 \text{ keV}$) must carry this caveat.

7.1 One table set for every altitude: density scaling

The tables are computed once, for Geant4 **G4_AIR** at $\rho_0 = 1.20479 \text{ kg/m}^3$ (dry air, 20 °C, 1 atm), the value the transport configuration records as **reference_density_kg_m3**; it is deliberately not called “STP”, because the density at which the RREA threshold and the reduced ion mobilities are quoted is a different one (Section 12), and every ratio in the code names its reference. But avalanches live at thundercloud altitudes where the air is several times thinner — so how can one table set be correct everywhere? The answer is that as long as the *composition* of the air is fixed (the N_2/O_2 ratio of the tables’ **G4_AIR** reference), every interaction rate is simply proportional to how many molecules a particle passes per meter. The engine therefore evaluates the local density ratio ρ/ρ_0 at each particle’s position and rescales on the fly: mean free paths stretch by ρ_0/ρ (thinner air = fewer collisions per meter), while drags and other inverse lengths shrink by the same factor. This rescaling is exact for every rate whose only density dependence is that molecule count. The lone exception is the Sternheimer density-effect term inside the collision stopping table, which stays frozen at its ρ_0 value: it is identically zero below $\approx 28 \text{ MeV}$ — hence exactly zero at the $\approx 1.7 \text{ MeV}$ friction minimum that sets the RREA threshold, so the threshold field itself rescales exactly — and reaches only $\approx 2.9 \%$ by 100 MeV, with the sign such that the drag is *under*-estimated.

Where ρ/ρ_0 comes from is a deck choice (**rrea.air_density_model**): either a single constant ratio for explicit diagnostic inputs, or a full altitude profile — **RreaAltitudeProfile** in the native library interpolates a CSV of density versus altitude, with the mesh’s z coordinate mapped to altitude above sea level. The canonical capture and all C&D comparisons use the realistic altitude profile, never a constant-density substitute. The same profile machinery can shape the imposed background field. The low-energy fluid follows suit: ion mobility is density-scaled with the same ratio, and the electron mobility and attachment are read from the source-anchored E/N closure table, whose reduced field carries that density, so the chemistry and conductivity are consistent with the transport.

7.2 Loading and integrity checks

The loader (`rrea::RreaInteractionTables`, native library) validates the tables semantically before transport mutates state: required files and columns, units and ranges, monotone energy/CDF grids, finite positive rates, normalized probabilities, cutoff agreement, and model identity. Any mismatch aborts the run. Off-table energies are an error, not a clamp. Schema-6 contains *no* calibrated fudge factors. After validation `Finalize()` resolves every table to a fixed `RreaTable` enum slot, so the hot path indexes an array rather than looking a name up.

7.3 Geant4-derived transport data and validation

A natural beginner’s question is why Geant4 is not called inside WarpX. Geant4 is a full transport framework with its own geometry, stepping, and threading model; embedding it in a PIC loop would be impractical. The project therefore samples a versioned schema-6 table set produced from a declared Geant4 reference. Geant4 itself never executes inside WarpX.

The only production data tree is `rrea_transport_tables/schema6/production/`, tracked in the repository as RREATBL containers (`native_amrex_rrea/include/rrea/RreaTableContainer.H`): each file is a JSON header naming every column with its dtype, over one deflate stream of byte-shuffled float64 columns. `scripts/rrea_table_pack.py` writes them and dumps any column back to CSV for review. Nothing is narrowed — a float32 store would collapse the sub-ulp energy anchors the loader requires to stay distinct. `transport_physics.json` stays plain text: it carries model identifiers, cutoffs, acceptance tolerances and the path of every table, which is where a replacement set is named.

At run time `RreaInteractionTables` loads the validated tables: a rate such as “mean free path of a 3.7 MeV electron against hard Møller scattering” is interpolated log-log between grid nodes, and when an interaction fires its final state is drawn by inverting the appropriate CDF.

8 The Low-Energy Fluid and the Field Solve

8.1 Fluid state

`rrea::LowEnergyFluidState` (native library) holds, per mesh cell: low-energy electron density n_e^{low} , positive and negative ion densities $n_{\pm}$, their source rates (fed by the Monte Carlo demotions and ionization), the conductivity σ , the local Maxwell relaxation time $\tau_m = \varepsilon_0/\sigma$, and the fluid charge density. Each adaptive material substep uses a symmetric half-reaction/transport–Maxwell/half-reaction split. The outer-step source-rate fields remain intact until the final half reaction, so their known integral is distributed exactly across the substeps; table attachment is re-evaluated after each updated field. The transport stage computes $\sigma = e \sum_s \mu_s n_s$ with mobilities scaled by air density, summed only over carriers that are actually advanced (a species counted in σ but held stationary would break charge continuity); and drift the carriers with adaptive shared, r -weighted finite-volume substeps using the committed donor-upwind face currents that enter the corresponding Maxwell substeps. The code aborts on any substep whose multidimensional outgoing carrier CFL exceeds the configured limit.

8.2 Why the field solve is different from stock WarpX

WarpX’s electrostatic solver answers “what field does this charge make?” The thundercloud problem needs more: an *imposed* background field (the cloud’s charging, held fixed as a boundary/reference state) that the plasma progressively screens. The RREA solver therefore replaces that electrostatic solve entirely with an explicit finite-speed update: the ambient field is built once as a curl-free background (Section 8.4) and the total field is then advanced by the RZ TM Maxwell equations (Section 8.3).

Screening is a consequence of that advance rather than a term in an operator. Where the air has become conductive, the fluid’s own committed face current opposes the local field and the field relaxes on the Maxwell time $\tau_m = \epsilon_0/\sigma$ because Ampère’s law says it must. Material continuity is a fail-closed outer-step gate and Gauss a monitor; neither applies a correction.

8.3 The Maxwell TM branch

Every current here is poloidal, so under $m = 0$ the complete retained field content is the TM triplet (E_r, E_z, B_θ) , advanced on a staggered Yee mesh by explicit leapfrog: E_r on radial faces, E_z on axial faces, B_θ on corners, so both curls land exactly where they are consumed with no averaging. The solver advances only the *scattered* field over the static thundercloud ambient; their sum is what chemistry, drift and the particle gather see — the fluid consuming the exact face fields, while the particle gather reads a cell-centered two-point face average (one extra half-cell-symmetric smoothing relative to the faces Ampère evolves). The source is $\mathbf{J} = \mathbf{J}_{\text{fluid}} + \mathbf{J}_{\text{kinetic}}$, where $\mathbf{J}_{\text{fluid}}$ is the fluid’s own committed donor-upwind face current — the same object that moves the carriers — so Ampère’s law and continuity cannot disagree about how much charge moved, and there is no separate σ term to double-count. The explicit coupling reports the maximum over its internal substeps of `em_max_dt_sigma_face_over_eps0`, the explicit diffusion number, and the reaction spectral number. Targets, hard guards and the allowed substep range are resolved from the run configuration. The count is recomputed from all three stiffness estimates each outer step, without changing the particle, video or checkpoint timestep. The energy ledger lives on the one discrete RZ metric of the staggered mesh (dual volumes, `RreaMaxwellTMRZ.H`): Joule work and the boundary term are bilinear means over each Ampère substep, the boundary term being the exact discrete remainder of the summation by parts (face-aligned Poynting products plus the Faraday work of the absorber-owned corners), so that in the leapfrog’s mixed-time form $\Delta U_E + \Delta U_B + \Delta t (P_{\text{Joule}} + P_{\text{out}}) = 0$ closes to round-off — the native Maxwell smoke pins that identity and exact closed-domain conservation. The reported ledger keeps B_θ^2 at the outer half-time, so its residual is the $O(\Delta t^2)$ magnetic-stagger term. The kinetic chord current is constant over the outer step. B_θ is re-staggered into the internal spacing and back to the outer $t - \Delta t/2$ convention before checkpoint/radio output. Boundaries are first-order Silver–Müller; the native Maxwell smoke measures their residual field energy after an outgoing pulse. The axis is regular by construction ($E_r(0) = B_\theta(0) = 0$, and E_z is never sampled on the axis).

Material charge continuity is enforced before the outer step is committed: the actual WarpX particle deposit plus persisted fluid charge at both time levels must close cell by cell against the time average of the exact substep face currents handed to Ampère’s law. The mixed norm is 10^{-10} relative with a binary64 cancellation floor of $8\sqrt{\epsilon_{64}}$ times the heaviest live macro charge, swept from the population each step: that quantum sets the scale of the resampling residue, and losing a whole macro would still be $\sim 10^7$ floors. Only after the check passes may the outer step be published. Gauss’s law is a monitor because the deliberately current-free seed injection leaves a fixed baseline defect; with

continuity closed, any change from it is numerical drift rather than a hidden source channel.

8.4 The imposed background and its lateral extent (radial taper)

The imposed field enters as a background potential ϕ_{bg} (`InitializePhi`, `RreaFieldInitializer.H`), from which *both* ambient face families are realized as differences of that one potential (`InitializeBackgroundState`, `RreaAmrexAdvance.cpp`). Because both come from the same ϕ_{bg} , the discrete $(\nabla \times E)_\theta$ telescopes to exact cancellation, so the ambient is curl-free by construction rather than by assertion — and the code verifies it at every interior B_θ corner, failing closed at initialization if it ever stops holding.

The background supports a **radial taper** (`rrea.field_taper_r_start_m / rrea.field_taper_r_end_m`):

$$\phi_{\text{bg}}(r, z) = \phi_{\text{ref}} + f(r) [\phi_z(z) - \phi_{\text{ref}}], \quad E_z = f(r) E_z^{\text{prof}}(z), \quad E_r = -f'(r) [\phi_z(z) - \phi_{\text{ref}}],$$

with f a C^2 quintic smoothstep ($f=1$ for $r \leq r_{\text{start}}$, $f=0$ for $r \geq r_{\text{end}}$) and ϕ_{ref} the midrange of ϕ_z over the domain (minimizing and symmetrizing the fringe E_r). Because the taper is applied at the *potential* level, the background stays curl-free, and the implied ρ_{ext} is a physical charge configuration: the profile’s charge layers laterally confined to $r < r_{\text{end}}$ plus the annulus fringe charge — a storm charge region of finite lateral extent instead of infinite slabs terminated by a charged wall. The wall column ($f=0$) is pinned at the constant ϕ_{ref} : a genuinely distant, field-free grounded boundary. In the taper annulus the ambient field can drop below the local density-scaled RREA threshold, allowing the avalanche front to stall before it reaches the wall.

The fade width is not a free choice — the circulation constraint. The fringe field is not an implementation detail: for *any* curl-free field that carries the full profile at small r and vanishes at large r , $\oint \mathbf{E} \cdot d\ell = 0$ forces the entire column potential across the fade annulus, so

$$\max |E_r| \simeq 1.875 \frac{(\max \phi_z - \min \phi_z)/2}{r_{\text{end}} - r_{\text{start}}}$$

(the 1.875 is the quintic smoothstep’s peak slope; changing the taper shape does not remove the circulation constraint). A usable fade width therefore satisfies $w \gtrsim 1.875 \Delta\phi_z / (2 E_{\text{acceptable}})$. The driver audits the fringe against the local density-scaled runaway threshold, aborting on any super-threshold pocket. The loaded vertical profile is already approximately zero at the domain floor and exactly zero above its upper support, so it needs no separate vertical taper.

9 Diagnostics, Video, and Checkpoints

Reduced CSV (`rrea_reduced.csv`) One row every `rrea.diag_interval` steps with core volume-integrated quantities and configuration-dependent extras: species populations and physical weights, fluid totals, conductivity extrema, field-fraction statistics (the `field_fraction_*_accel_region` columns span the profile-active band inside the taper core; a channel-local field needs the frame stream), per-channel interaction counters, and the energy/charge ledger totals — among them `em_charge_continuity_signed_accum_e`, the running signed continuity residue. Global-population, optical-source and gate diagnostics read this file.

Video diagnostic When enabled, the coupling resamples selected quantities onto a fixed $n_r \times n_z$ display grid at scheduled simulation times. The `rrea_video_frames_binary_v4` contract stores exactly five arrays, in order: energetic-electron density above the threshold, positive-ion density, $|E|$, photon density above the threshold, and positron density above the threshold. Frames append to `video_frames.bin` with CSV/JSON sidecars; readers fail closed if the format or exact five-array list is missing or changed. These are display resamples, not state, so v4 narrows them to `float32` and byte-shuffles and deflates each frame independently (`RreaVideoFrameCodec.H`, mirrored on the read side by `scripts/rrea_video_contract.py`). Compression is data-dependent, and the only precision loss is the declared `float64`-to-`float32` display conversion. Frames are therefore variable length, and the `byte_offset/byte_length` columns of the per-frame CSV are the index — which keeps the restart trim an exact truncation at a frame boundary, the property the cluster/laptop handoff depends on. Rendering to MP4 is a separate offline step (`scripts/render_rrea_profiled_video.py`); capture resolution is fixed at run time, but colormaps and styling are render-time choices. The standard video’s left profile axis uses one fixed scale for the initial radial mean (gray dashed), current radial mean (blue), and current first-cell “ $r = 0$ ” slice (red).

Checkpoint v2 Alongside each WarpX checkpoint, the coupling writes an `rrea_checkpoint/` payload: all fluid MultiFabs, the previous PIC charge state, the scattered Maxwell state, and per-rank RREA counters (including signed-plane diagnostic counters and append position). The checkpoint reader owns format, dimensions and state consistency: it verifies the completion marker, the schema and format versions, and the mesh/state dimensions of every field it reads. (Simulation time and step continuity are WarpX’s own; the RREA payload neither writes nor checks them.) A *rank-count* change requires refolding rank-local counters, so it aborts unless the run opts in with `rrea.allow_restart_epoch_change=1` and a reason string — what the driver’s `--restart-epoch-break` emits. Use that same marker for an intentional engine or physics-model change; it is also the sanctioned route for moving a checkpoint between the cluster and a laptop at a different rank count; the counters then fold deterministically onto the new layout; if the reduced-CSV header changes across the break the engine archives the superseded stream itself (`ArchiveCsvIfHeaderChanged`, `RreaCouplingDiagnostics.cpp`). Restarted runs are compared numerically; byte identity with an uninterrupted trajectory is not an acceptance condition. `docs/RREA_RERUN_GUIDE.md` carries the operator recipe.

10 From Simulation to Radio, Longitudinal-Field and Optical Observables

The engine outputs populations, fields, and currents. Offline analysis uses the Maxwell boundary record for direct off-axis free-space radio, an exact-axis longitudinal-field diagnostic, and the native ion-pair source for optical estimates.

10.1 EM fields from the recorded Huygens surface

A Maxwell capture appends the scattered (E_r, E_z, B_θ) on a closed surface **inset eight cells** from the absorber to `rrea_em_boundary.bin` every step, and `scripts/compute_rrea_em_radio.py` propagates it to an observer by a retarded surface integral (Love equivalence: $\rho_s = \epsilon_0 \hat{n} \cdot \mathbf{E}$,

$\mathbf{M}_s = -\hat{n} \times \mathbf{E}$, $\mathbf{J}_s = \hat{n} \times \mathbf{H}$). Three details are load-bearing. The surface must be inset, because on the boundary itself Silver–Müller slaves B_θ to the tangential E , so a record taken there carries no independent magnetic information. B_θ is a leapfrog half-step behind E , so it is interpolated on its own time axis ($t - \Delta t/2$); treating the two as simultaneous shifts the magnetic record by half an outer step. And the recorded field enters the kernel raw: $\rho_s = \varepsilon_0 \hat{n} \cdot \mathbf{E}$ cancels the $1/4\pi\varepsilon_0$ of the Jefimenko charge terms and $\mathbf{H} = \mathbf{B}/\mu_0$ cancels the μ_0 of the $\partial_t \mathbf{J}$ term, so neither constant appears in the integrand. Reinstating either scales that family of terms by $\varepsilon_0^{\pm 1}$.

Version 4 timestamps label the completed electric-field step. Legacy version 3 labels were one outer step ahead; the reducer and the loader of older observer CSVs remove that offset. Restart recovery converts retained version 3 frames to version 4 before appending, without altering their fields. The free-space exterior reconstruction also assumes source containment. The default PARMA column reaches the domain endcaps beyond the inset surface; currents in those exterior strips have not been accounted for by this integral. Their effect must be assessed before interpreting it as the total exterior field.

The on-axis output is not radio. For an $m = 0$ source E_r and B_θ vanish on the symmetry axis, so the Poynting flux there is identically zero. The exact-axis E_z this integral returns is a longitudinal Coulomb/induction field, which is what an overhead field mill or nadir field-change meter measures, not a propagating waveform. A radio prediction needs an off-axis observer and the full transverse vector integral with azimuthal quadrature; a distant on-axis waveform alone can therefore never establish a self-quenching claim.

10.2 Radio scope

For a nonzero observer offset, `compute_rrea_em_radio.py` reconstructs the full vector Love-equivalent surface sources around each RZ ring and performs azimuthal retarded quadrature. The $m = 0$ symmetry does not collapse that integral off axis: it makes the field values ϕ -independent, while the observer distance and hence the retarded time are not. The series is truncated to the window every quadrature node can answer from the record, because extrapolating past the last recorded frame would invent a boundary source and an artificial pulse tail. It writes $\mathbf{E}$, $\mathbf{B}$ and $\mathbf{S} = \mathbf{E} \times \mathbf{B}/\mu_0$ in free space. No ground image, ionosphere, terrain or antenna gain is applied. A geometric projection onto the ALOFT EFCM (the ER-2’s flat-plate electric-field-change meter) and nominal unit-gain fast/slow responses are instrument scenarios, not calibration. The evaluator stages the required chronological source brackets in one reused contiguous RAM slab per observer, retaining the global E/B time origins. The slab spans 4096 output samples plus their delay envelope, not the whole capture. The optional azimuth check compares N with $2N$ for both E and B; it reports refinement differences rather than declaring production convergence. For long captures, select observers separately with `EM_OBSERVERS` and size wall limits from measured loading and evaluation times. The launcher’s eight-hour default runs all three sequentially; it is not a 1 ms runtime guarantee. This IS the source of the multiband figure’s waveform and spectrum panels: a host-side reduction (`cluster/hpc/slurm_rrea_em_observers.sbatch`) writes one series per observer – a 100 km ground receiver, the ER-2 at 20 km MSL, and the nadir field mill on the exact-axis path – beside the band series, and only those small CSVs travel. It runs where the multi-GB record lives and is kept out of the render job, which is minutes rather than hours. The series carry OBSERVER time; the figure removes the geometric delay to put them on its source-time axis. The compact-source current-moment waveform stays as the dashed reference on the ground panels, and carries the panels outright for a capture whose reduction has not been run, which every panel

title names. Both paths see a *truncated* source: an electron absorbed at the top boundary stops contributing to $M(t)$, so an escape burst injects a spurious dM/dt ; `netchord_escaped_charge_e` bounds it.

Video diagnostics. `scripts/extract_rrea_band_series.py` reduces the frame stream on the capture host to band population series for the video and the multiband figure: both species everywhere above 12 km MSL over the full mesh radius. A whole-population observable needs a large volume, not a sample at one altitude, because a thin band sits in the macroparticle-occupancy tail and is shot-noise limited there. Each is an instantaneous population, not a crossing flux; a leading zero interval is a sampling-occupancy floor. The render job refreshes the sidecar before every render and drops a stale or missing strip rather than drawing it. The renderer also draws the optical source or irradiance series described below, but no radio waveform.

10.3 Optical emissions at 337.1 nm and 777.4 nm

`scripts/compute_rrea_optical_emissions.py` estimates the two ASIM/MMIA photometer bands from a profiled production capture. The framework is Xu, Celestin & Pasko, JGR 120, 1355 (2015; repo copy docs/papers/optical_emissions_Xu.pdf): the optical output of a TGF-scale avalanche is carried by the low-energy electron population, not the runaways. Two source terms:

Degradation term (baseline). Every ionizing cascade converts a fixed fraction of the deposited energy into fluorescence, independent of the ambient field — this is precisely the air-fluorescence yield of the cosmic-ray literature. For the 337.1 nm band (N_2 2P(0,0), $C^3\Pi_u \rightarrow B^3\Pi_g$) the world average is 7.04 ± 0.24 photons/MeV at 800 hPa, 293 K (Rosado et al., arXiv:1103.2022; the AIRFLY absolute measurement is 7.9 ± 2.0), with collisional quenching $FY(P, T) = Y_0/(1 + P/P'(T))$, $P'(293\text{ K}) = 15.9\text{ hPa}$ and $P' \propto T^{0.86}$. That exponent is $1/2 - \alpha$ for a collisional cross section $\sigma \propto T^\alpha$, with $\alpha_{337.1} = -0.36 \pm 0.08$ measured by AIRFLY (Ave et al., *Nucl. Instrum. Meth. A* **597**, 50, 2008, Table 1); the $\alpha = 0$ ($\sqrt{T}$) convention that paper refutes overestimates the yield at altitude by $\sim 8\%$ near 9 km. Water-vapour quenching is strong ($P'_{H_2O} = 1.28\text{ hPa}$) but negligible above $\sim 5\text{ km}$ and is not modelled. The optical production rate and its source-weighted altitude come from the engine’s native cumulative ion-pair source (`ion_pair_source_weight_cumulative` and `ion_pair_source_z_weighted_m_cumulative`); the script exits if a capture predates them. Multiplying that rate by $W_{\text{air}} = 33.97\text{ eV}$ gives the deposition rate. The script prints `transport_energy_loss_eV` and its implied energy per pair as a consistency diagnostic, not a second deposition estimate: tracked-secondary energy is booked when created and again where it deposits, while bremsstrahlung never enters the ion-pair tally. Soft bremsstrahlung below the cut is therefore also absent from the fluorescence driver; its share is printed alongside.

Field-driven term (a scenario, not a bound). Fluid electrons re-accelerated by the local field excite $N_2(C^3\Pi_u)$ directly (threshold 11.03 eV). Xu et al. tabulate the *total* yield per runaway electron per meter at ground pressure: 6.2, 10.7, 38.3 photons/m (300–430 nm) at $E/N = 16, 46.5, 69.9\text{ Td}$. Dividing by the field-free degradation floor ($dE/dx \cdot FY = 0.22\text{ MeV/m} \times 15.8\text{ ph/MeV} = 3.48\text{ ph/m}$, recomputed from the 337 nm yield model rather than hard-coded) turns their table into an enhancement factor $f(E/N)$ that the script log-interpolates and applies as the upper edge of the

337 nm band, with E/N from the n_e -weighted field-fraction column (`profile_active_abs_e_over_e0_peak_ne_low_weighted_r*`).

The two edges are *scenarios*, *not bounds*, and the error runs both ways. f is convex, so evaluating it at one mean E/N understates the mean of f over a field mixture (Jensen: a 50/50 mix of 0 and 69.9 Td gives 6.0, while $f(34.95 \text{ Td}) = 2.5$). Against that, the Xu term is switched *off* below Xu’s own lowest RREA point: for $E/N < 16 \text{ Td}$ the script returns $f = 1$ exactly — no invented 0 Td anchor, no extrapolation into the unconstrained segment below Xu’s table (pinned in `test_optical_physics.py`). Whether the term contributes is therefore a property of the capture’s source-weighted E/N history, not of a previously analyzed field amplitude. E/N also mixes an n_e -weighted average over the whole `profile_active` z -band with a point density at the beam mean altitude. When the screening column is absent the enhancement is not applied at all ($f = 1$) rather than assuming an unscreened field.

777.4 nm is not a nitrogen band. The red MMIA photometer line ($\text{OI } 3p^5P \rightarrow 3s^5S$) comes, in a cold discharge, from dissociative excitation of O_2 : $\sigma = 4.23 \times 10^{-18} \text{ cm}^2$ at 100 eV (Erdman & Zipf, JGR 92, 1987), i.e. $\sim 3.5 \times 10^{-3}$ excitations per ion pair ($x_{\text{O}_2} \sigma_{777} / \sigma_{\text{ion}}$, stated $\times 3$ uncertainty from the degradation-spectrum weighting), heavily quenched ($A = 3.69 \times 10^7 \text{ s}^{-1}$ against collisional quenching at $5.9/9.4 \times 10^{-10} \text{ cm}^3/\text{s}$ by N_2/O_2 — $\text{O}(3p^3P)$ TALIF proxies, Niemi et al. 2005; quench factor $\sim 8 \times 10^{-3}$ at 12 km). MMIA sees strong 777.4 nm from hot leader channels (Heumesser et al., GRL 48, 2021, report blue $\sim 3\times$ red for TGF pulses, with the red understood as leader emission), and this simulation contains no leader; its 777.4 nm output is therefore a cold-discharge estimate rather than a leader-emission prediction.

Optical analysis tools. The CLI produces a three-panel figure (lightcurve + ionization driver, source photon rates with the uncertainty bands, and isotropic-source irradiance at the configured observer, whose range for an overhead observer is derived from the capture’s emission-weighted source altitude) plus a companion `.optical.csv` with key intermediates (pair rate, altitude, pressure, quench factors, E/N , enhancement factor, and band edges). The video renderer imports the same `optical_series` calculation for its optical strip.

Radiative transfer to the observer. Modelled, and on by default. `scripts/rrea_cloud_scattering.py` hands the source series to **CloudScat** (the ALOFT campaign’s Monte Carlo radiative-transfer code; Luque et al., *Geosci. Model Dev.* **13**, 5549, 2020), kept as an untracked clone in `external/cloudscat_cpp`. The helper owns the upstream URL, build and physical self-test; no repository document pins a separate revision. The coupling uses CloudScat’s own `source.type: "list"`: each reduced-CSV sample becomes a sub-source $\{[0, 0, z(t_i)], Q_i = r(t_i) \Delta t_i, t_i\}$, so the emission altitude follows the beam and no linearity or fixed-position assumption is needed. CloudScat returns $\text{photons m}^{-2} \text{ s}^{-1}$ *per emitted source photon*, so the observed flux is that timeline times $\sum_i Q_i$; its next-event estimator already carries the direct term through $e^{-\tau}$, so the result *replaces* $1/(4\pi d^2)$ rather than correcting it. Normalisation is pinned by a closed-form self-test: with no droplets and Rayleigh disabled the time-integrated fluence must equal $\sum_i Q_i/(4\pi d^2)$ within the self-test tolerance.

The size of the correction depends steeply on how deep the source sits below cloud top, which is therefore a flag (`--cloud-top-km`) carried on every figure title rather than a buried constant. Against a top at or below the emission altitude the source is effectively outside the liquid-water

cloud, the direct beam survives, and both the lag and the FWHM change little. A source buried many Mie mean free paths below the top instead loses its direct beam and develops a delayed, broadened tail. The simulation contains no cloud state: any quoted lag, width or peak must name the assumed top and be recomputed from that capture. Note there is no inequality between the scattered and free-space fluences: scattering redirects photons that would otherwise have missed the observer entirely, so the cloudy value legitimately exceeds the bare line-of-sight one. Because the figures share one source-time axis across gamma, optical and radio, much of that tail falls past the right-hand edge; the panel title states what fraction of the scattered fluence the window actually holds. `--no-cloud-scattering` restores the free-space-only panel.

Stated limitations. A single mean emission altitude per time sample (beam mean z), which suppresses the cascade’s vertical quench and transport variation; no hot-leader channel, so the 777.4 nm trace is a cold-discharge estimate; the 777.4 nm per-pair yield is a 100 eV cross-section ratio, not a degradation-spectrum integral ($\times 3$ band on the figure). Sub-cutoff radiated energy is never re-deposited as ion pairs (§7); the script prints that share alongside the ledger check, and it bounds how much the yields are low.

11 Model Scope and Known Model-Form Uncertainties

The production model is an *axisymmetric* (RZ , $m = 0$), *zero geomagnetic field* RREA model with one supported field equation: finite-speed Maxwell TM (Section 8.3). The following limitations are model-form or numerical uncertainties.

- **Geometry and fields.** RZ $m = 0$ cannot represent lateral avalanche displacement, filamentation or azimuthally localized feedback. The self-consistent B_θ IS gathered to the charged particles and applied by a Boris rotation (`RreaChargedInterleave.H`); what is omitted is the geomagnetic field, which a horizontal component would need $m \neq 0$ to represent. That omission need not be small: a representative $50 \mu\text{T}$ geomagnetic field gives a 1-MeV gyroradius of ~ 95 m, comparable to an avalanche length. This is a model-form limitation of the $m = 0$ geometry, not an assumed-small correction.
- **Temporal discretisation.** The charged transport is first order in the macro step: the microchord drifts on the pre-kick direction, hazards and drag are frozen per segment, and Goudsmit–Saunderson deflection is applied at the chord end. Quantitative rates therefore need a timestep ladder in the same physical regime; a bound from an earlier source or field history is not automatically transferable. This is a numerical-resolution knob and needs no new table bundle.
- **Low-energy fluid closure.** Production uses the dry-air `en_table_flux_v1` mobility/diffusion tensor and `en_table_attachment_v1` attachment from `electron_flux_mobility_attachment_diffusion.csv`. The table is interpolated in E/N and supplies flux mobility, longitudinal/transverse diffusion, and a source-anchored dry-air attachment rate. Positive and negative ions retain lumped reduced mobilities of $1.4/1.9 \times 10^{-4} \text{ m}^2/\text{Vs}$. Electron–ion and ion–ion recombination are separate reaction channels; detachment is implemented but deliberately zero. The v3 table joins Milloy’s zero-field mobility and Hegerberg–Reid drift data to de Urquijo dry-air transport through 100 Td; Morrow–Lowke supplies separate

density-linear/two-body and density-squared/three-body attachment above the measured 3–30 Td three-body band. Its positive 10^{-6} -Td endpoint is the measured zero-field asymptote. The remaining model-form uncertainty is room-temperature dry-air data and two lumped ion species in cold cloud air; above 100 Td is forbidden because the fluid has no field-ionisation channel.

- **Macroparticle resolution.** Source weights and the controller target are run inputs. Exact cell-charge constraints can leave the live count above target when no support group has removable null-space capacity, so the hard ceiling permits those sparse supports without corrupting deposited charge; sampled interactions therefore enter the nonlinear $n_e \rightarrow \sigma \rightarrow E$ loop in finite-weight packets. Quantitative screening/collapse claims therefore require a population-target convergence ladder.
- **Transport cutoffs.** Bremsstrahlung photons below the table configuration’s tracked cut are continuous drag: their backscatter, upstream feedback seeding and displaced deposition are absent (largest exposure: the escaping spectrum below the cut and soft-photon-derived optical inputs). The run records the continuous soft-bremsstrahlung energy so this missing tracked-photon share can be measured directly; it is radiated away rather than re-deposited as ion pairs. Hard-Møller secondaries below the table configuration’s threshold are restricted stopping: straggling, displaced knock-ons and their runaway-promotion probability are unconverged. The *transport* cutoff needs no new bundle: the tables are general across cutoffs (grid floor 1 keV), so a cutoff ladder runs on one bundle and one executable. The sub-cutoff band can re-accelerate into the runaway population, so the cutoff is a feedback-path parameter and not merely a cost knob. The selected production bundle makes the default charged cutoff **1 keV**, its certified charged-energy floor. An override cannot reach this bundle’s 50 keV hard-Møller threshold: the engine aborts unless the charged cutoff is strictly below the table configuration’s `hard_moller_secondary_threshold_eV`, which is 50 keV, because hard secondaries born at or below the cutoff would be counted once as continuous drag and once as a dropped particle. Do not confuse this with photon cutoff, which is derived as 50 keV from the table configuration’s `brems_absolute_cutoff_eV`: they are different cutoffs on different species. A charged cutoff at or above 50 keV therefore needs a bundle whose hard-Møller threshold sits above the new value. The two `N*_ge_100keV_real` columns report the surviving near-cutoff band on every run; use them to size and interpret the ladder in that run’s field.
- **Positron annihilation timing.** A positron falling below the transport cutoff is thermalized and annihilated in the same step: there is no thermal positron lifetime and no positronium. The omitted lifetime is density- and positronium-dependent and can be long compared with a timestep or an avalanche e-folding. The approximation can therefore advance annihilation-photon production and must be tested when photon timing or feedback is the observable.
- **Radial taper.** The taper is a declared finite storm-charge model, not a boundary trick; its fringe is fully simulated. The driver audits each resolved field against the local runaway threshold, and recorded energetic tails can enter the taper, so the fringe cannot be dismissed as unsampled.
- **Observables.** Optical irradiances are scenarios conditioned on the assumed cloud top and on the emission model; 337 nm’s Xu enhancement applies only where the realized source-weighted E/N reaches its tabulated domain, and 777.4 nm remains a leaderless cold-RREA estimate. The standard multiband aircraft panels use the recorded dM/dz side-car to remove the axial point-dipole error, but still collapse each radial current distribution to the axis. The remaining

radial model error is captioned on every figure; samples outside the supported retardation window are blanked rather than drawn. The separate off-axis Huygens calculation, which the waveform panels now draw, is the direct finite-source free-space observable. Independently of geometry, the EFCM prediction is conditional on the flat plate’s along-heading orientation (Kochkin et al. 2021): at exact nadir the instrument reads zero whatever the source does, and the nadir prediction lives in the step read by the LIP field mill instead. At a 100 km ground receiver the axial compact-source error is instead small, $(L/R)^2 \approx 0.005$ for the production domain height, but the radial extent sets a corner $c/(2\pi \text{rms}_r \sin \theta) \approx 20$ kHz above which the collapsed dipole misses the broadside phase spread of the ~ 2 km-wide current: against the surface integral it agrees to 2% below 10 kHz and overestimates the 30–300 kHz band four-fold.

Not modelled: Xu’s discriminating band ratio. Xu et al.’s headline diagnostic is the $2\text{PN}_2/1\text{NN}_2^+$ intensity ratio, which separates production mechanisms (their Table 2: 0.74, 1.10, 5.06 for RREAs at 4.3, 12.5, 18.8 kV/cm, against 1.58 for thermal runaway at a leader tip and 101 for the streamer zone). Computing it needs the 391.4 nm 1NN_2^+ band, which this pipeline does not produce. That is a deliberate scope choice, not an oversight: the target instrument is ASIM/MMIA, whose photometers are 337 nm, 777.4 nm and 180–230 nm, and 391.4 nm is not among them. A comparison against Xu’s mechanism ratio — or against a ground/TARANIS 391.4 nm photometer — would require adding that band first. The 180–230 nm LBH channel is likewise not modelled.

12 Checking the Physics: Avalanche Multiplication vs. Coleman–Dwyer

Focused tests establish specific implementation properties; they cannot prove that the model matches nature. For RREA physics the community yardstick is the *avalanche length* λ : the distance over which the runaway electron population multiplies by a factor e . Coleman and Dwyer distilled large independent Monte Carlo campaigns into one empirical formula,

$$\lambda_{\text{CD}} = \frac{7300 \text{ kV}}{E_0 - E_{\text{th}} \rho},$$

where E_0 is the applied field, E_{th} the effective runaway threshold, and ρ the air density *relative to the density at which E_{th} is quoted*. That reference is 1.29230 kg/m^3 (dry air, 0°C , 1 atm), not the table reference of Section 7.1 and not a sounding’s own surface density. It is fixed independently of any simulation: Dwyer’s break-even field, 218 kV/m, is where the field force equals the minimum total collision drag, and NIST ESTAR puts that minimum at $\approx 1.66 \text{ MeV cm}^2/\text{g}$ for dry air, so the two are consistent only near 1.31 kg/m^3 . Anchoring E_{th} instead at a sounding’s own surface density raises the threshold, and that error both shrinks the above-threshold interval and narrows the margin inside it, so it suppresses the predicted multiplication. Two threshold conventions circulate in the literature — $E_{\text{th}} = 276 \text{ kV/m}$ and 284 kV/m — so this project benchmarks against *both* and reports a descriptive $\pm 10\%$ consistency band for each. These comparison labels are not software pass/fail labels.

The benchmark must compare like with like. Reduced constant-field and mono-particle probes test components but are not C&D results. Every campaign comparison runs through `scripts/run_rrea_profiled_video_capture.py`, with the real altitude profiles and compact video diagnostics still enabled. The dedicated source is a cohort of simultaneous, unit-weight electrons on a fixed source

disk (cohort size `CD_INITIAL_SEED_COUNT` in `scripts/rrea_cd_physics.py`, disk radius `channel_radius_m` in `config/rrea_defaults.json`). Their energies are deterministic stratified quantiles of the 7.2 MeV exponential, conditioned to the certified table range; independent realizations use distinct transport RNG seeds.

Population peaks and time-series peak/trough ratios are not multiplication observables. The engine instead accumulates cumulative upward and downward electron weight at physical planes, and the offline report forms the signed time-integrated flux

$$\Phi_{\text{net}} = \Phi_{\uparrow} - \Phi_{\downarrow}, \quad M = \Phi_{\text{net}}(z_{\text{end}})/\Phi_{\text{net}}(z_{\text{source}}),$$

so equal-state recrossings cancel. The primary selection is *locally runaway capable*: the gathered field magnitude exceeds the same energy- and density-dependent continuous drag used by transport. This force-balance test is deliberately independent of crossing direction; a directional $-\mathbf{E} \cdot \hat{\mathbf{v}}$ gate would count the upward leg while discarding the matching downward recrossing. The all-tracked channel is retained as the exact topological cancellation audit, and $K \geq 1$ MeV supplies a fixed energetic-census comparison.

The common physical interval starts at the injection altitude and ends at the first upper crossing of $|E| = 276n$ kV/m. Both denominator choices use those same endpoints:

$$S_T(z) = \int_{z_{\text{in}}}^z \frac{E(z') - Tn(z')}{7300 \text{ kV}} dz', \quad T \in \{276, 284\}.$$

Profile knots and crossings are integrated exactly. Endpoint compatibility and a post-equilibration spatial fit are reported separately; fields that do not meet the configured equilibration and plane-coverage criteria remain endpoint-only.

The exact interval above is the analytic context of the quoted profile table, but the diagnostic planes sit half a cell inside its two ends, so a measured flux ratio is compared with the same integral taken between those plane positions. A measured flux is never extrapolated to an unobserved endpoint, and a supplied plane list never chooses its own interval.

The implemented campaign is an early-time all-physics comparison: photon and positron transport, charged secondaries, the fluid and the self-consistent field remain enabled. The short observation window can reduce the contribution from long feedback loops, but it is not a feedback switch; high-field excess multiplication may be genuine feedback and is reported as a deviation rather than used to recalibrate transport. A complete realization is the statistical unit, and cohorts with different seed statistics are not pooled.

Exact fields, durations, cohorts and eligibility rules live in `scripts/rrea_cd_physics.py`; the planner consumes those values and `scripts/report_rrea_coleman_dwyer_production.py` performs the reduction. This section defines the physical observable, not a reference result.

13 Fail-Closed Operation and Focused Validation

A theme runs through everything above: *when in doubt, abort*. The same policy at every layer:

- source preparation aborts if WarpX’s loop shape changed;

- startup aborts on any species, table semantics, model-string, or cutoff mismatch;
- `RreaInteractionTables::Load`, the one owner of table semantics, validates the structures and values the engine actually consumes;
- transport aborts on optical-depth guard violations and ledger (charge/energy conservation) violations;
- when enabled outside the C&D campaign, `adaptive_resample_v1` thins the population toward `rrea.population_target_electron_macros` within a hysteresis band. Within each common order-1 CIC support, stochastic null-space weight moves preserve all four deposited cell charges exactly in every realization while keeping $E[w]$ for every particle. The controller plans globally over valid survivors plus staged charged candidates and preflights both plans, the stored-precision CIC moments, the physical ledgers and the three ceilings before either mutates a live weight, on deterministic global support groups so capacity and the geometric floor are decomposition-invariant. Exact CIC preservation means the nonlinear charge–conductivity–field loop receives no resampling charge impulse or correction current; particle and spectral observables stay stochastic, unbiased in their weight-linear estimates and bounded in variance only by a macroparticle ladder (§11);
- the checkpoint reader refuses an inconsistent payload and a detected rank-count or reduced-schema change without an explicit restart epoch;
- focused native smokes and `scripts/test_*.py` print an obvious PASS/FAIL; the module index (§14) names the pin of every physical process. Independent anchors include the ESTAR stopping-power comparison in `native_amrex_rrea/src/RreaScatteringSamplerSmoke.cpp` and the primary-source fluid-closure checks in `scripts/verify_fluid_closure_anchors.py`. C&D results run the real profiled production path; reduced probes remain component tests.

14 Physics-Module Index

Repository requirement (CLAUDE.md): every physical process lives in ONE named module with its governing equation or model citation at the top, reviewable in isolation and pinned by a native smoke or committed test. This index is the map for reviewing them one by one: process — module — pin. Native smokes are ctest targets under `native_amrex_rrea`; Python pins live in `scripts/test_*.py`. Two pins below are neither: `rrea_warpx_actual_continuity` and `rrea_warpx_transport_apply` are injected into the prepared WarpX tree by `scripts/prepare_rrea_warpx_source.py` and need a built `app_rz`, so they are build-scale, not minutes-scale.

Kinetic transport

- Chord geometry (cylinder exit, seam) — `RreaTransportPath.H` + `native_amrex_rrea/src/RreaTransportPath.cpp` — `rrea_transport_path_smoke`
- Gauss–Legendre quadrature (8-node, and the 2-node rule that is exact for a cubic) — `RreaGaussLegendre8.H` — `rrea_gauss_legendre8_smoke`

- Geometric density path ($\tau = \sigma_{\text{stp}}(E) \int \chi d\ell$ with $\chi = \rho/\rho_0$; midpoint χ per in-domain piece, relative error $\leq \frac{1}{24}(h/H)^2$; depth-to-position inversion exact inside a piece and additive across the seam) — `density_path_m / fraction_at_density_path_m / max_uniform_subcycle_density_path_m` (`RreaMaterialPath.H`) — `rrea_transport_path_smoke + rrea_charged_interleave_smoke`
- Line-integral field work along a microchord ($W = q \int_a^b \mathbf{E} \cdot \hat{d} d\ell$ and the mean $\mathbf{E}, \mathbf{B}$ from the same nodes; 2-node Gauss–Legendre per in-domain piece, exact because the gather is bilinear and the chord affine; residual bounds stated in the header) — `path_field_work_and_average` (`RreaPathFieldWork.H`) — `rrea_gauss_legendre8_smoke` (cubic exactness) + `rrea_charged_interleave_smoke` (the half-kick consumer)
- Field half-kick (Strang) + rotation — `RreaApplyChargedFieldHalfKick` (`RreaChargedInterleave.H`) + `RreaLorentzPush.H` — `rrea_charged_interleave_smoke` (work-only energy, zero floor, charge-sign reversal)
- Per-channel hazard rate (gate, then $1/\lambda = 1/\text{mfp}$, else exactly zero) — `RreaChannelInverseLength` (`RreaChargedInterleave.H`) — `rrea_charged_interleave_smoke` (exactness and short-circuit of the closed gate)
- Hazard optical depth + inversion — `RreaResolveRemainingSegmentHazard` (defined in `RreaParticleInteraction.H`, driven from `RreaChargedInterleave.H`) — `rrea_charged_interleave_smoke`
- Event-interleave sequencing (refreeze, null events) — `RreaRunChargedEventInterleave` — `rrea_charged_interleave_smoke` (mutation-checked)
- Substep skeleton — `RreaRunChargedSubstepInterleave` — same
- Continuous drag + ion-pair partition (the cutoff point on the chord by exact inversion of the same piecewise density path) — `RreaPlanContinuousDragSegment` → `RreaContinuousDragSegmentPlan` + `RreaPartitionContinuousIonPairs` (`RreaChargedInterleave.H`) — `interleave smoke`
- Local-energy ion-pair deposit ($N = wE/W_{\text{air}}$, $W_{\text{air}} = 33.97$ eV, ICRU 31) — `RreaIonPairWeightFromLocalEnergy` (`RreaParticleInteraction.H`) — `rrea_secondary_group_smoke`
- GS elastic application (one unit for all charged paths, RNG-key policy) — `apply_gs_elastic_scatter` (`RreaTransportStep.H`) — the ANGLE SAMPLER it calls is pinned by `rrea_scattering_sampler_smoke`; its application by `rrea_warpx_transport_apply`
- Charged discrete channels — phase-space gates, frozen competing rates, and complete hard-Møller / hard-Bhabha / bremsstrahlung final states (sampled kinematics, parent post-state, secondary, material sources, group closure; the tracked-brems channel opens only above the photon transport cutoff, for both species) — `RreaChargedChannelOpenAt`, `RreaChargedCompetingStpRates`, `RreaApplyHardMollerFinalState` / `RreaApplyBhabhaFinalState` / `RreaApplyBremsFinalState` (`RreaChargedFinalStates.H`) — `rrea_charged_interleave_smoke` (kinematics, brems recoil) + `rrea_secondary_group_smoke` (conservation)
- Photon discrete channels — Compton, photoelectric absorption, nuclear and triplet pair production, each booking its energy flow through the one owner of the expected split — `RreaApplyComptonFinalState` / `RreaApplyPhotoelectricFinalState` / `RreaApplyPairFinalS`

tate (RreaPhotonFinalStates.H) consuming RreaComptonPhotonEnergyFlow / RreaPhotoelectricPhotonEnergyFlow / RreaPairPhotonEnergyFlow (RreaParticleInteraction.H) — rrea_secondary_group_smoke + rrea_scattering_sampler_smoke

- Annihilation booking (in flight / at rest; Heitler final state from the tables) — spawn_positron_on_annihilation (RreaTransportStep.H) — rrea_scattering_sampler_smoke
- Demotion gate + state policy — RreaEvaluateElectronStatePolicy (RreaParticleInteraction.H) — secondary-group closure smoke
- Newborn residual advance (a secondary born at parent-chord fraction f flies the remaining $(1 - f)\Delta t$ on the PARENT RNG stream through the same drag, GS and final-state modules) — RreaNewbornAdvance.cpp with RreaNewbornElapsedStepFraction (RreaParticleInteraction.H), delegating to RreaRunChargedSubstepInterleave / RreaRunChargedEventInterleave — rrea_secondary_group_smoke + rrea_rng_endpoint_smoke
- Terminal/escape ordering — RreaResolveChargedTransportTerminal — interleave smoke
- Charged pre-push prologue / escape closure / survivor writeback (one owner for both charged species; charge sign explicit) — TransportStep::BeginRunningChargedParticle / CommitChargedEscape / CommitSurvivorChargedWriteback (RreaTransportStep.H) — rrea_warpx_actual_continuity + rrea_warpx_transport_apply ctest cases (both species transported)
- Secondary-group charge/energy ledger — RreaParticleInteraction.H group validation — rrea_secondary_group_smoke
- Transport transaction log (ordered replay for material sources, charged deposits, the Compton/photoelectric/pair records and positron annihilation; associative scalar tallies in a per-chunk RreaEventLedger folded once by RreaFoldEventLedger) — RreaTransportLog.H + RreaEventLedger.H — rrea_secondary_group_smoke + rrea_warpx_transport_apply
- Counter-based RNG streams — RreaRng.H / RreaUniformStream.H — rrea_rng_endpoint_smoke
- Electron transport phase (running chord, hazards, drag, GS deflection) — TransportStep::ApplyElectronTransport (RreaElectronTransport.cpp) — rrea_warpx_transport_apply + rrea_warpx_actual_continuity
- Photon transport phase (attenuation, channel draw, interaction point by exact inversion of the density path, secondary promotion) — TransportStep::ApplyPhotonTransport (RreaPhotonTransport.cpp) — rrea_warpx_transport_apply + photon-channel sampler smokes
- Positron transport phase (charged loop + annihilation booking; separate from the electron loop by policy) — TransportStep::ApplyPositronTransport (RreaPositronTransport.cpp) — rrea_warpx_transport_apply (both species transported)
- Secondary transaction commit (collective charge/energy closure of every staged group, then the newborn advance and the terminal writeback) — TransportStep::DrainAndCommitSecondaries (RreaSecondaryCommit.cpp) — rrea_secondary_group_smoke + rrea_warpx_actual_continuity

- Per-step Monte Carlo entry point (builds the `TransportStep` context, runs the four phases in order) — `RreaParticleInteraction::Apply` (`RreaParticleInteraction.cpp`) — `rrea_w arpx_transport_apply`

Interaction tables and samplers

- Load + semantic validation + enum-indexed access (one `RreaTable` id per table, resolved once by `Finalize()`) — `RreaInteractionTables::Load` (the one owner of table semantics) — `rrea_interaction_tables_smoke`
- RREATBL container decode (header gate, named columns, un-shuffle, finiteness) — `RreaTableContainer.H` — `rrea_table_container_smoke` + a Python round-trip pin
- Log-log rate interpolation (no clamp/extrapolate) — `RreaInteractionTable::Interpolate` — endpoint/midpoint pins plus the charged-channel quarterpoint holdout replay
- Photon channel partition — `SamplePhotonChannel` — pinned vs `PhotonProcessInverseLengths`
- GS angle sampler (endpoints are BOUNDS) — `sample_gs_angle_cos` — sampler smoke + Geant4 slab benchmarks
- Screened-Rutherford single elastic — analytic-quantile sampler — pinned to closed form
- Wentzel-VI branch + 100 MeV transition — scattering-contract units — one-ulp transition pin
- Brems quantiles + bounded Modified-Tsai angle — conditional CDF + `SampleBremsstrahlungCosTheta` — energy holdouts, truncated-density moments, endpoint support and e-/e+ event closure in `rrea_scattering_sampler_smoke`
- Møller/Bhabha secondary CDF — per-species samplers (endpoints $T_{cut} \dots E/2$) — sampler smoke + holdout
- Pair fraction + final-state bank replay — loader/validator + `SamplePairFinalState` (replay, validates the loader) — norm, coverage, $\langle \gamma \theta \rangle$, charge-symmetry pins
- Photoelectric subshell + Sauter-Gavrila angle — subshell sampler + angle unit — closure, K-edge, angle pins
- Compton (Butcher-Messel / Klein-Nishina) — `RreaInteractionTables::SampleComptonKleinNishinaFinalState` — analytic endpoint and KN-moment pins
- Annihilation spectrum (Heitler) + CM→lab — spectrum and final-state unit pins
- Stopping/drag accessors + density-ratio scaling (every rate exactly linear in ρ/ρ_0 ; the STP bundle is frozen once per refreeze and multiplied by the geometric density path) — thin table reads, linearity stated at Load — exact per-channel linearity pins

Field solve

- Faraday update (RZ TM) — `MaxwellAdvanceB` (`RreaMaxwellTMRZ.H`) — `rrea_maxwell_tm_rz_smoke`
- Ampère update + material current — `MaxwellAdvanceE` — same (exact $\tau = \varepsilon_0/\sigma$ decay)
- Silver–Müller boundary — `RreaMaxwellTMRZ.H` — measured residual in the smoke
- Axis regularity + CFL gate — solver loops + `RreaAmrexAdvance.cpp` gate — `rrea_maxwell_tm_rz_smoke` (axis) + `rrea_amrex_advance_smoke` (the two-dimensional Yee CFL refusal)
- Discrete RZ metric (cell/dual volumes, r-weighted divergence) — `MaxwellTMRZ` members (`RreaMaxwellTMRZ.H`) — closed-domain exact energy conservation + Gauss pins in the same smoke
- Ambient construction (curl-free taper) — `RreaFieldInitializer.H` (ϕ /taper) + `InitializeBackgroundState` (`RreaAmrexAdvance.cpp`; face realization, curl $\equiv 0$ verified fail-closed at every start, then committed as the $t = 0$ field) — advance smoke + the Python $|f'|_{max}$ identity pin
- Total-current assembly (fluid + kinetic face current in one concatenated allreduce per substep) — `AssembleTotalFaceCurrent` (`RreaAmrexAdvance.cpp`) — continuity gate exercises it
- Continuity reference priming (the seed batch’s own charge enters ρ^n before any particle moves, so an injection is not read as a continuity violation) — `PrimeMaxwellContinuityReference` (`RreaAmrexAdvance.cpp`) — `rrea_amrex_advance_smoke`
- Adaptive symmetric material substeps + continuity gate (+ charge-resolution floor) — `AdvanceOneStep` selects from conduction, diffusion and reaction stiffness, applies half-reaction/transport–Maxwell/half-reaction, and time-averages the shared currents for `VerifyMaterialContinuity` — fluid/advance smokes (+mpi4)
- Energy ledger (dual-volume energy, bilinear-mean Joule, exact discrete boundary term, ambient cross-term) — `MaxwellFieldEnergy` / `MaxwellMixedMagneticEnergy` / `MaxwellJoulePower` / `MaxwellBoundaryPoyntingPower` (`RreaMaxwellTMRZ.H`), assembled in `AdvanceOneStep` — `rrea_maxwell_tm_rz_smoke` (open-step identity to round-off)
- Gauss monitor (never a gate) — `RecordGaussMonitor` on `DivergenceRZ` — Maxwell smoke Gauss pin
- Huygens surface record + scattered-state persistence — `RreaMaxwellTMRZRecord.H` — native record/restart smoke, production-hook clock check and reader-side layout pins in `test_delivery_physics.py`; the checkpoint reader owns the mesh check
- Cell-centred commit and total-field gather (the gather forms its cell corners from the replicated Maxwell state by the same two-point face average the commit uses, so gathered = committed to the last bit) — `CommitMaxwellTotalField` / `GatherTotalFieldRZ` (`RreaAmrexAdvance.cpp`) — `rrea_amrex_advance_smoke` (bilinear identity, odd-radial axis scale)
- Particle gather stencil — `RreaFieldGatherStencil.H` — `rrea_field_gather_stencil_smoke`

- Cell-centred-to-native field handoff (nodal directions average the two bracketing cells; only physical boundaries clamp) — `InterpolateCellCenteredToNative` (`RreaFieldHandoff.H`) — `rrea_low_energy_face_transport_smoke`
- Air density vs altitude (the ratio every rate, drag, attachment and mobility is rescaled by, §7.1) — `RreaAltitudeProfile.H/RreaAltitudeProfile.cpp` — `rrea_altitude_profile_smoke`

Low-energy fluid

- Chemistry channels (exact-exponential attachment, pair-loss ODE, detachment) — `LowEnergyFluidState` update, per-channel ODE comments — `rrea_low_energy_fluid_smoke`
- `en_table` closure evaluation (E/N , hold-below, abort-above), evaluated once per substep at the end of the fluid/Maxwell substep — `RreaAmrexAdvance::EvaluateClosureFields` driving `BuildElectronClosureCellFields` + the closure formulas in `LowEnergyFluidState.H` — face-transport + closure-table smokes
- Drift face flux (donor upwind) + diffusion tensor — `BuildExplicitFaceTransport` — `rrea_low_energy_face_transport_smoke` (+`mpi4`)
- FV update + positivity/outflow limiter — `ApplyExplicitCarrierTransport` — same (limited-fraction diagnostics)
- Conductivity assembly — `ComputeConductivity` — fluid smoke
- Mobility law (`absolute` vs `reduced_stp_density_scaled`, and the conversion between the table density ratio ρ/ρ_0 and the reduced-mobility ratio ρ/ρ_L) — `MobilityLaw` / `resolve_mobility_law` / `checked_density_ratio` (`LowEnergyFluidState.H`), with $\rho_L = n_L M_{\text{air}}/N_A$ derived in `RreaConstants.H` and a hard abort on an unknown law string — `rrea_low_energy_fluid_smoke` + `rrea_low_energy_face_transport_smoke`
- `en_table` closure *bundle* anchoring (zero-field–100 Td mobility, diffusion and two-/three-body attachment) — `scripts/generate_rrea_fluid_closure_bundle.py` + `scripts/verify_fluid_closure_anchors.py` — primary-source anchor check over the shipped CSV
- Vacuum/subnormal rule — `LowEnergyFluidState.H` (bounded-freeze statement) — denormal transport tests
- Reaction charge audit — in-update audit (throws at 512ε) — fluid smoke

Deposition and population

- RZ zigzag segment deposition + physical metric + escape — `RreaSegmentDepositionRZ.H` — analytic case suite + the production continuity gate
- CIC thinning + global survivor/candidate transaction plan — `RreaCicConservingResample.H` + `RreaCouplingPopulation.cpp` — CIC smoke + one-/four-rank continuity fixture; capacity/floor in the reduced CSV

- Population action selector + adaptive split (each split moves exactly half the parent weight to a clone, so $E[w]$ and the deposited cell charges are preserved) — `RreaAdaptiveResampleAction` / `RreaAdaptiveResampleSplitOutcome` (`RreaRng.H`), applied in `RreaCouplingPopulation.cpp` — `rrea_rng_endpoint_smoke`
- RZ shell volume $V = \pi(r_{\text{out}}^2 - r_{\text{in}}^2)\Delta z$, both radii clamped to the axis — `rrea::rz_shell_volume` (`RreaConstants.H`), the one owner for the reduced observables, the event accumulator, the fluid face transport, the Maxwell metric and the video frame — `rrea_event_accumulator_smoke` + `rrea_reduced_observables_smoke`
- Seed injection window/momentum — `RreaSeedSchedule.H` — `rrea_seed_schedule_smoke`
- PARMA continuous cosmic-ray e^-/e^+ source ($q_s = \phi_s/H$, $\dot{N}_s = A_{\text{cm}^2} \langle \int \phi_s dE \rangle_z$ per species, macros allocated $\propto \dot{N}_s$; Sato 2015/2016, non-commercial JAEA scope; fixed 2019-05-27 solar reference and the shared $e^\pm$ angular law are stated model limitations) — `scripts/rrea_parma_source.py` (+ the thin Fortran evaluator `scripts/parma_electron_query.f90` against the vendored `rrea_parma_upstream/` model code and data; schedule `species_id` routes each event to its WarpX container in `InjectScheduledSeeds`) — `test_parma_source_physics.py` + `rrea_seed_schedule_smoke`
- Strict ceilings (fail-closed backstops) — collective preflight in the coupling — `rng-endpoint` band pins
- Plane-flux observable — `RreaPlaneFlux.H` — transport-path smoke + report-side pins
- Event-source shell-volume normalization + off-box deposit routing (per-thread scratch FABs folded into the fields once per step by `FoldScratchIntoFields`, then beyond-ghost deposits routed to their exact owning cells in one collective flush) — `RreaEventAccumulator.H` + `native_amrex_rrea/src/RreaEventAccumulator.cpp` — `rrea_event_accumulator_smoke`
- Video-frame binning (the five display arrays resampled onto the fixed $n_r \times n_z$ grid on the same shell volume, then float32/shuffle/deflate) — `RreaCouplingVideoFrame.cpp` + `RreaVideoFrameCodec.H` — `scripts/test_driver_restart_safety.py` (the v4 stream contract)

Observables and analysis (Python unless noted)

- Field-based reduced observables (E/N percentiles, field fractions, ohmic/conduction moments) — `RreaReducedObservables.H` (C++) — `rrea_reduced_observables_smoke` vs analytic fields
- Species moments ($M(t)$, dM/dz , energy census) — coupling species sweeps (WarpX-bound) — `rrea_segment_deposition_smoke` pins deposited current moment = $qw\Delta r/dt$ against the net chord
- Retarded Hertzian-dipole terms (D&C eq. 12 is the companion **B**) + charge moment — `dipole_field_terms` / `cumulative_charge_moment` — EFCM-null + LIP-step pins
- Finite-source line dipole — `finite_source_fields` — runtime 1% conservation gate
- On-axis retarded surface kernel — `on_axis_longitudinal_fields` (`compute_rrea_em_radio.py`) — symmetry, causality and kernel pins in `test_delivery_physics.py`

- Off-axis Love-equivalence field — `off_axis_fields` — interior static point charge vs exact Coulomb, plus symmetry/causality/convergence pins
- Instrument responses (RC high-pass single owner, digitizer low-pass, boxcar single owner, Butterworth bands, spectra) — `compute_rrea_radio_waveform.py` + multiband units — DC-gain, Parseval pins
- EFCM plate + LIP field mill — `efcm_plate_field` / `lip_vertical_field` — exact-nadir-null + step-survival pins
- Optical yields (FY337 quench, Xu enhancement, 777.4 chain) — `compute_rrea_optical_emissions.py` units — closed-form yield pins + no-extrapolation pin
- Cloud transport coupling — `rrea_cloud_scattering.py` (`scattered_flux`, config builder) — committed physical self-test + absolute-clock pin
- C&D avalanche integral $N_\lambda = \int (E - E_{th}\rho)/7.3\text{ kV } dz$ — `estimate_integrated_avalanche_lengths` — closed-form slab pin in `test_cd_science.py`
- Signed plane multiplier + cohort deviations (both thresholds) — report/analysis physics functions — pinned in `test_cd_science.py`
- Ambient-field audit + Td identity — `audit_ambient_field` (`rrea_profiled_atmosphere.py`) — production-margin + identity pins
- Band series (population, not flux) — `extract_rrea_band_series.py` — occupancy caveat stated; panels labeled

15 Repository Map

Path	Contents
native_amrex_rrea/src, native_amrex_rrea/include/rrea/ upstream/WarpX-26.06, upstream/AMReX-26.06	the native AMReX library (fluid, solver, tables, seeds, accumulators) protected-fork gitlinks that own the exact dependency commits
native_amrex_rrea/warpx_overlay/ Source/Rrea/ scripts/prepare_rrea_warpx_source.py	the WarpX glue (coupling singleton, MC transport engine, RNG) builds the combined WarpX source tree (copy + marker-guarded patches)
rrea_transport_tables/ cases/rrea/production/ config/rrea_defaults.json	tracked, packed schema-6 Geant4-anchored tables plus <code>transport_physics.json</code> , the sole semantic configuration; they travel with a clone and <code>hpc.pysync</code> runner-owned production input deck single source for run-defining defaults (population control, capture geometry/schedule, HPC layout, run profiles)
EFIELD_profile/, scripts/rrea_profiled_atmosphere.py scripts/rrea_run_support.py scripts/run_rrea_profiled_video_capture.py, scripts/render_rrea_profiled_video.py scripts/compute_rrea_em_radio.py, scripts/compute_rrea_optical_ emissions.py, scripts/extract_rrea_band_series.py, scripts/rrea_cloud_scattering.py cluster/hpc/	the digitized field and NRLMSISE density/threshold CSVs, and the shared loader that maps mesh z to altitude (Sections 7.1, 8.4) shared capture orchestration science-run driver and offline video renderer the offline observables pipeline (Section 10)
docs/	site-agnostic Slurm build/run/render launchers and the campaign planner; site identity stays in private local configuration live architecture, rerun, and HPC operation guides

Build flow in one line: *pinned pristine WarpX + overlay via `prepare_rrea_warpx_source.py` $\rightarrow$ CMake (RZ, MPI, OpenMP, -O3) $\rightarrow$ `warpx.rz` executable + separately built native smoke executables.*

16 Glossary

Avalanche length λ Distance over which the runaway population multiplies by e ; benchmarked against the Coleman–Dwyer formula under both threshold conventions (Section 12).

Relativistic feedback Dwyer’s mechanism: backward-propagating bremsstrahlung photons and pair-produced positrons from one avalanche seed new avalanches at the source region; above a threshold field the discharge becomes self-sustaining. Emerges naturally here from tracked photon/positron transport.

Macroparticle / weight One computational particle standing for w real particles; all its interactions are sampled once and applied at weight w .

MultiFab AMReX’s distributed mesh field (one physical quantity on all boxes of all ranks).

PIC Particle-in-cell: particles for the kinetic species, a mesh for fields, coupled by deposit/gather.

Optical depth $\int n \sigma d\ell$ along a path; the expected number of interactions. Sampling “the next event at optical depth $-\ln u$ ” is the standard MC transport trick used per channel.

Demotion Converting a below-cutoff charged secondary from a would-be particle into fluid density sources (charge- and energy-conserving).

Density scaling Every table rate is rescaled at run time by the local ρ/ρ_0 (mean free paths $\propto \rho_0/\rho$, drags $\propto \rho/\rho_0$), from either a constant ratio or an altitude profile. **Two different reference densities are in play** — the tabulation reference and the one the RREA threshold and reduced ion mobilities use — and they are not interchangeable; Section 7.1 states both.

Maxwell TM solve The supported finite-speed RZ field update for (E_r, E_z, B_θ) with kinetic and fluid face currents (Section 8.3).

Continuity/Gauss Material continuity is a fail-closed outer-step gate; the Gauss defect is a monitor. Neither applies a correction.

Schema-6 The versioned interaction-table generation shipped here: Geant4-anchored and semantic-validation checked.

Fail-closed Any integrity doubt terminates the run; nothing is repaired, clamped, or defaulted silently.

Ceiling A hard cap on macroparticle count per species. Under the production driver’s `adaptive_resample_v1` it is a fail-closed backstop above the control band; under `strict_abort_v1` reaching it is a collective abort. Never a silent drop (§13).

Capture One production run of `scripts/run_rrea_profiled_video_capture.py` and the directory it writes — reduced CSV, frame stream, checkpoints and side-car series — which every offline pipeline in §10 reads.